Go2 EDU 分层建图 + 爬楼导航演示操作手册

> 目标：让一个**不了解代码细节**的操作员照着这份手册从零开始，完成两层地图建图、路点录制，并在现场只按几次键完成"楼下 → 爬楼 → 楼上"的全自动导航演示。

---

0. 目录

- 1. 系统概述
- 2. 硬件与网络前置
- 3. 代码部署与编译
- 4. 一次性准备：建两张地图 + 录路点
- 5. 正式演示（全程只按 6 次键）
- 6. 实时监控指标
- 7. 按键速查表
- 8. 调参手册
- 9. 日志与文件产出
- 10. 故障排查

---

1. 系统概述

本方案在 Unitree `keyDemo` 的基础上自研 `keyDemo3`：

- 用 Unitree 官方 SLAM 负责楼下/楼上两张 PCD 地图的建图与重定位导航
- 楼梯过渡区由 `keyDemo3` 直接驱动 Go2 EDU 的 SportClient 盲走爬楼，其中用 **Cross-Track Error（CTE）闭环**保证直线爬楼、消除初始朝向偏差
- 路点管理采用显式流：`s` 存内存、`S` 显式落盘到 json、`d` 先 reload 再执行，避免内存/磁盘数据不一致
- 终端输出全程 tee 到带时间戳的 `keyDemo3_*.log`，便于事后回看

演示一次要按的键一共 6 次：`a → 1`，`d`，`c`，`c`，`a → 2`，`d`。

---

2. 硬件与网络前置

项	值
机器狗	Go2 EDU / Developer
激光雷达	Livox Mid-360（或 XT16，驱动换成 `xt16_driver`）
计算单元	Go2 拓展坞（192.168.8.3）
演示主机到拓展坞 SSH	`ssh unitree@192.168.8.3`（公钥免密）
Go2 通信网卡	`go2eth`

目录约定：

端	路径
开发机（本地 PC）	`/home/jiangtao/huazhijian/project/old_go2/unitree_sdk2/`
Go2 拓展坞（dock）	`/home/unitree/jiangtao/unitree_sdk2/`
Unitree 官方部署	`/unitree/module/unitree_slam/`（只读，`keyDemo` 官方二进制在这里）

> **重要**：`/unitree/module/unitree_slam/` 是 root 所有的只读部署。`keyDemo3` 部署在 `/home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/`，和官方互不干扰。

---

3. 代码部署与编译 （修改代码前先 copy 到自己的目录下）

首次部署（已完成，仅记录步骤供复查）：

```
# 1. 本地同步源码到 dock
scp unitree_slam/example/src/keyDemo3.cpp \
    unitree@192.168.8.3:/home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/src/

# 2. dock 上 aarch64 编译
ssh unitree@192.168.8.3
cd /home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/build
cmake ..
make keyDemo3 -j
```

产物位置（dock 上）：`/home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/build/keyDemo3`。

后续修改后的增量编译：`cd .../example/build && make keyDemo3`，只要 2 秒。

---

4. 一次性准备：建两张地图 + 录路点

这一步只需要做一次，之后正式演示直接复用。

4.1 启动底座服务（3 个终端 ssh 到 dock 开 3 个窗口）

终端 1：启动 SLAM 服务

```
cd /unitree/module/unitree_slam/bin
./unitree_slam
```

终端 2：启动 Mid-360 雷达

```
cd /unitree/module/unitree_slam/bin
./mid360_driver
# 若是 XT16 雷达: ./xt16_driver
```

终端 3：启动 keyDemo3（我们自研的按键工具）

```
cd /home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/build
./keyDemo3 go2eth
```

终端 4（可选）：RViz2 可视化，只在建图/对点时开

```
cd /unitree/module/unitree_slam/rviz2
rviz2 -d mapping.rviz
# 重定位阶段改用 relocation.rviz
```

> **关键注意：按 `w` 保存 PCD 之前必须关闭 RViz2**，否则 PCD 落盘会失败或超时。拓展坞资源有限，建议建图过程中也不开 RViz2，需要看效果时临时开一下再关。

4.2 建一楼地图

把狗放在"一楼演示起点"，面朝一楼目标方向放稳。

在终端 3 的 keyDemo3 里：

步骤	按键	动作 / 期望输出
1	`q`	开始建图，终端输出 `startMapping statusCode:0` 和 `Successfully started mapping.`
2	—	用遥控器控制狗把一楼全部区域走一遍，最后**停在楼梯入口、正对楼梯**
3	（关闭 RViz2）	如果开了 RViz 要先关
4	`w` → 输入 `1`	存成 `/home/unitree/floor1.pcd`。期望看到 `Save pcd successfully` 和 `>>> Mapping session closed.`（keyDemo3 会自动清理 slam node）

4.3 建二楼地图

手抱/遥控把狗运到二楼起点（楼梯顶部着陆点附近），面朝二楼目标方向放稳。

步骤	按键	动作 / 期望输出
1	`q`	开始新建图会话。由于 4.2 已 `stopNode`，这次不会报 `Exceeding the maximum value range`
2	—	遥控走完二楼所有路径
3	（关闭 RViz2）	同上
4	`w` → 输入 `2`	存成 `/home/unitree/floor2.pcd`

4.4 录一楼路点

把狗放回一楼起点。

步骤	按键	说明
1	`a` → 输入 `1`	加载 floor1.pcd 做重定位，`currentFloor = 1`
2	—	观察终端，等 `statusCode:0` 和 `info:"Successfully started relocation."`，确认 curPose 不再是全 0
3	`s`	记录当前位姿到内存（提示 `size=1`，`dirty=true, press 'S' to persist.`）
4	—	遥控走到下一个路点
5	`s`	记录下一个点（size=2, 3, ...）
6	...	重复走 + `s`
7	**最后一个点**要录在**楼梯入口、正对楼梯**的位姿	这个点是爬楼前的停车点，它的朝向 yaw 会被 keyDemo3 当作"楼梯方向"，非常关键
8	`S`	**大写 S** 显式落盘，终端打印 `[S] floor1 (N pts) -> /home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/build/f1.json [OK]` 才算成功

4.5 录二楼路点

`a` → `2` 切到二楼，重复 4.4 的 3–8 步。第一个点一般是"楼梯口刚爬上来的着陆点"，最后一个点是二楼终点。录完再按 `S`，看到 `[S] floor2 (M pts) -> .../f2.json [OK]`。

至此，两张地图 + 两条路点 list 持久化到磁盘。下次演示开机即可用。

---

5. 正式演示（全程只按 6 次键）

把狗放回一楼演示起点，面朝目标方向。

三个终端启动服务（和 4.1 完全一样）：

```
# 终端 1
cd /unitree/module/unitree_slam/bin && ./unitree_slam

# 终端 2
cd /unitree/module/unitree_slam/bin && ./mid360_driver

# 终端 3
cd /home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/build
./keyDemo3 go2eth
```

keyDemo3 启动时会自动 `loadTaskListFun` 把 f1.json / f2.json 读回内存，你会看到：

```
Loaded floor1 N pts from f1.json
Loaded floor2 M pts from f2.json
```

确认两个数字和你录的一致。

然后在终端 3 依次按键：

#	按键	发生什么
1	`a` → `1`	加载 floor1 重定位，狗在原点附近 ICP 收敛
2	`d`	keyDemo3 从 `f1.json` reload + 开始依次导航到每个路点。终端打印 `[d] Reloaded floor 1 (N pts) from disk, starting navigation...`
3	—	等狗自动走完一楼路点，最后停在楼梯口正对楼梯
4	`c`	开始 CTE 闭环盲走爬楼。终端每 0.5 秒打印 `[climb] dx=.. dy=.. offs=.. yerr=.. vyaw=..`
5	—	观察狗爬楼，到二楼起点后
6	`c`	停止爬楼，狗进入 BalanceStand，SLAM 恢复
7	`a` → `2`	切楼上地图，狗在二楼起点附近重定位
8	`d`	执行二楼路点，狗走到二楼终点

演示结束。

6. 实时监控指标

爬楼过程中终端会持续打印

```
[climb] dx=3.452 dy=0.008 offs=-0.004 yerr=-0.002 vyaw=-0.003
```

字段	含义	期望值
`dx`	已沿楼梯方向走了多少米	随时间单调增长
`dy`	当前横向偏离参考直线多少米（左正右负）	**|dy| ≤ 0.05 m** 是理想，收敛到一个小常数
`offs`	CTE 算出的 desired_yaw 偏移	典型几毫弧度
`yerr`	当前 yaw 与 desired_yaw 的差	接近 0 表示跟得住
`vyaw`	实际下发的转向速度	接近 0 表示走得稳

爬楼完成后查看

- 重定位终端回执 `statusCode:0`， `Successfully started relocation.`
- 若是 `errorCode:509 ICP low`： 说明狗不在二楼起点附近，ICP 初值离真实位置太远，需人工扶回起点再按 `a → 2`

---

7. 按键速查表

键	动作	备注
`q`	开始建图	仅准备阶段用
`w`	结束建图 + 存 PCD（选 1 / 2）	自动 `stopNode`； `currentFloor` 归零
`a`	加载地图 + 重定位（选 1 / 2）	后续 s/d 绑定这个楼层
`s`	把**当前位姿**记到内存 task list	**dirty=true，必须 S 才能落盘**
`S`	把内存 task list 写到 json（**覆盖**）	打印绝对路径
`d`	执行当前楼层 task list	dirty 时拒绝；不 dirty 时先 reload 再执行
`f`	清空当前楼层 task list（内存）	也会变 dirty
`c`	爬楼 toggle：开始 CTE / 停止	仅在狗对准楼梯、靠近楼梯口时按
`z` / `x`	暂停 / 恢复 SLAM 导航	
其他键	触发 `stopNodeFun` 兜底	误按会结束 slam node

---

8. 调参手册

所有参数都是 keyDemo3.cpp 里 `public float` 成员，改完重新 `make` 2 秒见效。

参数	默认	什么时候调	调整方向
`climb_vx`	0.35 m/s	爬楼太慢 / 太快撞台阶	推荐 0.2 ~ 0.5
`K_y`	0.5 rad/m	狗对横向漂移反应迟钝	调大（如 0.8）；震荡就调小
`K_psi`	1.5	yaw 跟踪迟缓	调大
`vyaw_max`	0.4 rad/s	转向过猛	调小
`max_yaw_offset`	0.52 rad (30°)	偏差大时不希望猛转	调小到 0.2 (11°)
`align_limit`	0.0873 rad (5°)	初始偏差大，希望预对齐多转一点	调大到 0.175 (10°)
`align_tol`	0.03 rad (1.7°)	预对齐收敛不到位	调小到 0.01

源码中位置：

```
/home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/src/keyDemo3.cpp
     约 140 行 public: 成员
```

修改并同步到 dock：

```
# 开发机本地改完
scp unitree_slam/example/src/keyDemo3.cpp \
    unitree@192.168.8.3:/home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/src/

# dock 上增量编译
ssh unitree@192.168.8.3 \
  'cd /home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/build && make keyDemo3'
```

---

9. 日志与文件产出

产物	位置（dock）	说明
一楼点云地图	`/home/unitree/floor1.pcd`	按 `w` → `1` 生成
二楼点云地图	`/home/unitree/floor2.pcd`	按 `w` → `2` 生成
一楼路点	`.../example/build/f1.json`	按 `S` 生成，**绝对路径**会打印到终端
二楼路点	`.../example/build/f2.json`	同上
运行日志	`.../example/build/keyDemo3_YYYYMMDD_HHMMSS.log`	每次启动自动创建，终端所有输出 tee 到此
Unitree 服务日志	`/unitree/module/unitree_slam/bin/logs/`	官方 slam_server / gridmap / planner 日志

查看本次爬楼的详细过程：

```
cd /home/unitree/jiangtao/unitree_sdk2/unitree_slam/example/build
ls -lt keyDemo3_*.log | head -1        # 最新一份
less -R keyDemo3_20260422_172600.log   # -R 让 less 正常显示 ANSI 颜色
grep '\[climb\]' keyDemo3_*.log        # 只看爬楼控制循环输出
```

---

10. 故障排查

10.1 按 `d` 看到红色 "UNSAVED changes"

```
[d] Floor 1 has UNSAVED changes in memory (5 pts). Press 'S' to save first, or 'f' to discard.
```

原因：按 `s` 加了点但忘记 `S`。修法：先按 `S` 看到 `[OK]`，再按 `d`。

10.2 按 `q` 第二次建图报 "Exceeding the maximum value range"

这是 FAST-LIO 的局部 cube 还绑在上一次建图的原点。**keyDemo3 已经修了**：按 `w` 会自动 `stopNode`。如果你用的是**原版 `keyDemo`** 就会遇到；统一改用 `keyDemo3` 即可。

10.3 按 `a` 报 `errorCode:509 "ICP low"`

狗当前位置 / 姿态与期望的初始位姿（默认 `(0,0,0, q_w=1)`，即建图起点）差太远。修法：

- 把狗推回对应楼层的**建图起点**再按 `a`
- 或者录路点时把第一个点就录成"起点"，每次 `a` 之后立刻 `d` 从起点开始走

10.4 按 `w` 存 PCD 超时报 `statusCode:3202`

多半是 RViz2 还开着占用 IO。关掉 RViz2 再按 `w`。

10.5 按 `c` 报红色 `No rt/sportmodestate samples received yet`

Go2 的 sport service 和 keyDemo3 没连上 DDS。检查：

- 启动 keyDemo3 时的网卡参数是否正确（本手册用 `go2eth`）
- `/unitree/module/unitree_slam/bin/unitree_slam` 是不是正常跑着
- 遥控器能否正常控制狗；不能的话先修 DDS

10.6 爬楼偏航严重（dy 持续扩大）

看 `keyDemo3_*.log`：

- `dy` 单向增大到 > 0.3 m：说明 `K_y` 太小，CTE 拉不回。把 `K_y` 从 0.5 调到 0.8 试试
- `dy` 来回震荡：`K_y` 或 `K_psi` 太大，调小
- 爬楼起步 1~2 s 就大偏差，之后稳住：正常（CTE 稳态误差），只要 `|dy| < 0.05 m` 就没事

10.7 爬完按 `a → 2` 重定位不收敛

楼上地图 `floor2.pcd` 建图起点和狗实际爬完楼的落点偏差太大（> 0.5 m）。修法：

- **建 floor2 地图时从"狗爬完楼自然停下来的那个点"开始**（按 q 那一刻就是楼上地图原点）

- 或者在 `keyDemo3.cpp` 的 `case 'a'` 里把默认初始 pose `poseDate{}` 改成实测的楼顶落点偏移（不推荐，改代码不如改建图起点）

附录：工作流概念图

```
┌── 一次性准备 ──────────────────────────────────┐
│                                                 │
│  q → 走楼下 → w→1      建 floor1.pcd            │
│  q → 走楼上 → w→2      建 floor2.pcd            │
│                                                 │
│  a→1 → s×N 录路点 → S  持久化 f1.json           │
│  a→2 → s×M 录路点 → S  持久化 f2.json           │
│                                                 │
└─────────────────────────────────────────────────┘

┌── 每次演示 ────────────────────────────────────┐
│                                                 │
│  a→1 → d            一楼自动导航到楼梯口        │
│  c                  开始 CTE 爬楼               │
│  c                  到楼顶停                    │
│  a→2 → d            二楼自动导航到终点          │
│                                                 │
└─────────────────────────────────────────────────┘
```

文档版本：v1.0 / 2026-04-22
对应代码：`keyDemo3.cpp` 1002 行